

Analisi Algoritmo Genetico come Feature Selection su Dataset DARWIN

Baris Giorgio

Fionda Andrea
Petrilli Davide

Massaro Federico

Università degli Studi di Cassino e del Lazio Meridionale
Corso di Laurea Magistrale LM-32

A.A. 2025/2026

Indice

1	Introduzione e specifiche del problema	1
1.1	Introduzione	1
1.2	Contesto e specifiche del problema	2
1.2.1	Dataset DARWIN	2
1.3	Obiettivi	3
2	Implementazioni	3
2.1	Algoritmo genetico	4
2.2	Funzione di fitness	4
2.3	Logger	5
3	Analisi dei risultati	5
3.1	Scenario Dimensione Popolazione	6
3.1.1	Risultati ottenuti	6
3.2	Scenario operatori genetici	10
3.2.1	Risultati ottenuti	10
3.3	Scenario criteri di stop	11
3.3.1	Risultati ottenuti	11

1 Introduzione e specifiche del problema

1.1 Introduzione

L'obiettivo di questa tesina è analizzare l'efficacia di un algoritmo genetico come metodo di feature selection su un dataset reale, in questo caso il dataset DARWIN. La feature selection è una fase cruciale nel processo di costruzione di modelli predittivi, poiché consente di identificare le caratteristiche più rilevanti per la previsione, migliorando così le prestazioni del modello e riducendo il rischio di overfitting. Gli algoritmi genetici sono ispirati ai processi di evoluzione naturale e sono ampiamente utilizzati per risolvere problemi di ottimizzazione complessi. In questo contesto, utilizzeremo un algoritmo genetico per selezionare un sottoinsieme di caratteristiche dal dataset DARWIN, al fine di migliorare le prestazioni di un modello di classificazione.

1.2 Contesto e specifiche del problema

Un team di ricerca deve analizzare l'impatto dei diversi parametri degli Algoritmi Genetici GA sulla loro performance e convergenza utilizzando il dataset DARWIN. Questo dataset contiene caratteristiche di scrittura a mano per lo studio dell'Alzheimer, offrendo un contesto reale e significativo per l'analisi degli algoritmi genetici.

1.2.1 Dataset DARWIN

Il dataset DARWIN utilizzato in questa tesina è composto da 174 campioni, ciascuno con diverse features che rappresentano vari aspetti della scrittura a mano. La struttura del dataset include:

- **Features di scrittura a mano:** In questa categoria rientrano le metriche che descrivono l'atto della scrittura come evento discreto e la sua intensità di contatto.
- **Features di cinematica:** In questa categoria rientrano le metriche che descrivono la dinamica del movimento, distinguendo tra quando la penna scrive e quando si sposta in aria (hovering).
- **Pressione della penna:** In questa categoria rientrano le metriche che indicano la forza fisica esercitata dal soggetto sulla superficie durante la scrittura.
- **Parametri geometrici:** In questa categoria rientrano le metriche che descrivono l'ingombro spaziale e le proporzioni del testo o del disegno prodotto.
- **Caratteristiche temporali:** In questa categoria rientrano le metriche che analizzano la gestione del tempo e le pause, fattori cruciali per identificare rallentamenti nei processi motori e cognitivi.

e presenta le seguenti complessità:

- **Multiple feature categories:** Il dataset non contiene un unico tipo di misura, ma riflette diverse dimensioni del comportamento umano (motorio, cognitivo e fisico). Un Algoritmo Genetico deve selezionare un "mix" equilibrato di queste categorie perché, una selezione sbilanciata solo su parametri temporali, potrebbe ignorare micro-tremori cinematici fondamentali per la diagnosi. Questo implica che la funzione di fitness deve essere in grado di valutare l'apporto informativo di variabili con unità di misura e significati fisici completamente diversi tra loro.
- **Dati numerici continui:** A differenza di dataset categorici (es. "Sì/No", "Rosso/Verde"), le feature di DARWIN sono numeri reali (es. `mean_speed` può essere 125.45, `pressure_mean` può essere 0.00045). Alcune variabili hanno range enormi (migliaia di unità per il tempo), mentre altre hanno range infinitesimali (decimali per l'accelerazione o il jerk). Senza un adeguato pre-processing (come la normalizzazione), le feature con valori numerici più grandi potrebbero "schiacciare" quelle con valori piccoli, traendo in inganno l'algoritmo. Per il GA, questo significa che il "segnale" utile è nascosto in una precisione numerica elevata, rendendo difficile distinguere il rumore statistico dalla reale rilevanza clinica.
- **Correlazione complessa tra le features:** Le features del dataset non sono indipendenti l'una dall'altra. Questo indica l'alta possibilità di ridondanza: Se `"total_time"` è alto, è molto probabile che anche `"air_time"` o `"paper_time"` siano alti. Se il GA seleziona tutte e tre queste features sta creando ridondanza nella selezione delle features. Selezionare feature ridondanti aumenta la complessità del modello senza aggiungere potere predittivo. Il GA affronta questo problema attraverso una funzione di fitness multi-obiettivo che penalizza la ridondanza interna.

1.3 Obiettivi

L'obiettivo di questa tesina è quello di implementare un algoritmo genetico per la feature selection sul dataset DARWIN e analizzare l'impatto dei diversi parametri dell'algoritmo. In particolare, ci concentreremo su:

- **Convergenza dell'algoritmo**
- **Stabilità della soluzione finale**
- **Velocità di esecuzione**
- **Robustezza della selezione**

tenendo conto dei seguenti vincoli imposti:

- **Utilizzare stesso seed per avere confronti equi**
- **Effettuare minimo 30 run per ogni configurazione**
- **Rispettare un tempo massimo di esecuzione per ogni run**
- **Gestire appropriatamente i dati mancanti o anomali**

Per raggiungere questi obiettivi, implementeremo un algoritmo genetico che include:

- **Codifica binaria per la selezione delle features**
- **Funzione di fitness basata su correlation analysis**
- **Operatori genetici modulari**

e una funzione di logging dettagliata per monitorare l'andamento dell'algoritmo e facilitare l'analisi dei risultati, che ci restituisce le seguenti informazioni:

- **Fitness per generazione**
- **Feature selezionate**
- **Diversità della popolazione**
- **Tempo di esecuzione per generazione**

Queste implementazioni verranno valutate su tre scenari differenti:

- **Scenario Dimensione Popolazione**
- **Scenario Operatori Genetici**
- **Scenario Criterio di stop**

che verranno descritti più dettagliatamente nella sezione dedicata all'analisi dei risultati per ogni scenario. L'analisi dei risultati ci permetterà di identificare le configurazioni ottimali per l'algoritmo genetico in questo contesto specifico e di trarre conclusioni sulle migliori pratiche per la feature selection su dataset complessi come DARWIN.

2 Implementazioni

In questa sezione descriveremo le implementazioni effettuate per lo svolgimento della tesina, evidenziando le scelte progettuali e i dettagli tecnici.

2.1 Algoritmo genetico

L'algoritmo genetico implementato per questa tesina segue la struttura classica degli algoritmi evolutivi, con alcune personalizzazioni specifiche per il problema di feature selection sul dataset DARWIN. Di seguito sono descritti i principali componenti dell'algoritmo (classe "GeneticAlgorithm"):

- **__init__**: Inizializza i parametri dell'algoritmo, come la dimensione della popolazione, il tasso di crossover, il tasso di mutazione e il numero massimo di generazioni.
- **initialize_population**: Genera una popolazione iniziale casuale di soluzioni, dove ogni soluzione è rappresentata da un vettore binario che indica quali features sono selezionate. L'uso di "np.random.seed" assicura che la popolazione iniziale sia identica in ogni esecuzione del test.
- **evaluate_population**: Il metodo evaluate_population esegue la fase di assegnazione della fitness. Attraverso un'iterazione sulla popolazione, viene richiamata la funzione fitness_correlation_based per misurare la qualità di ogni set di feature selezionato. Questo punteggio guiderà le successive fasi di selezione e riproduzione dell'algoritmo.
- **calculate_population_diversity**: Il metodo calculate_population_diversity quantifica la variabilità genetica del sistema. Attraverso il calcolo della varianza media dei bit componenti i cromosomi, l'algoritmo monitora il livello di esplorazione dello spazio di ricerca. Tale metrica è fondamentale per diagnosticare fenomeni di saturazione o di eccessiva omogeneità della popolazione.
- **run**: Il metodo run implementa il ciclo principale dell'algoritmo genetico. Attraverso l'integrazione di elitismo, operatori di crossover e mutazione, il sistema evolve la popolazione verso una selezione ottimale di feature. Il processo include un meccanismo di early stopping basato sulla convergenza stocastica, garantendo un equilibrio tra efficacia della ricerca e tempo computazionale.

2.2 Funzione di fitness

La funzione di fitness implementata si basa sul framework CFS (Correlation-based Feature Selection). Il punteggio assegnato a ogni individuo non dipende solo dalla capacità predittiva delle singole variabili ($\overline{r_{cf}}$), ma penalizza attivamente la ridondanza interna ($\overline{r_{ff}}$). Matematicamente, il numeratore premia l'associazione con il target, mentre il denominatore agisce come fattore di penalità per la collinearità tra le feature selezionate. Per ottimizzare le prestazioni computazionali, le matrici di correlazione vengono pre-calcolate e interrogate tramite indicizzazione booleana durante l'evoluzione del GA. La formula utilizzata è la seguente:

$$\text{Fitness} = \frac{k \cdot \overline{r_{cf}}}{\sqrt{k + k(k-1) \cdot \overline{r_{ff}}}} \quad (1)$$

dove:

- k è il numero di feature selezionate
- $\overline{r_{cf}}$ è la media delle correlazioni tra le feature selezionate e la variabile target
- $\overline{r_{ff}}$ è la media delle correlazioni tra le feature selezionate

Il codice è composto da 3 funzioni principali:

- **_point_biserial_**: Misura la relazione tra una variabile continua (feature) e una variabile binaria (target), utilizzando **_corr_cache** per non ricalcolare mai la stessa correlazione più di una volta.

- **fitness_correlation_based:** Riceve l'individuo (il set di feature attive) e le matrici pre-calcolate. Estrae solo le correlazioni relative alle feature "attive" nel cromosoma e calcola la media triangolare superiore della matrice di correlazione per ottenere $\bar{r}_{ff}$ senza duplicati. Applica la formula di Hall per restituire il valore di merito come punteggio di fitness.
- **precompute_correlations:** Calcola le matrici di correlazione (r_{cf} e r_{ff}) una sola volta prima dell'avvio del GA. In questo modo evita migliaia di calcoli ridondanti durante l'evoluzione, velocizzando drasticamente l'esecuzione.

2.3 Logger

La funzione di logging implementata è progettata per monitorare in modo dettagliato l'andamento dell'algoritmo genetico durante le sue iterazioni. Il logger raccoglie e memorizza informazioni chiave che permettono di analizzare la performance e il comportamento dell'algoritmo nel tempo. La classe "ExperimentLogger" include i seguenti metodi:

- **__init__:** Inizializza le strutture dati interne:
 - **self.runs:** Una lista destinata a contenere i record dettagliati di ogni singola esecuzione.
 - **self.feature_counts:** Un dizionario (hash map) utilizzato per tracciare la frequenza di selezione delle feature. In un contesto di feature selection, questo permette di identificare quali variabili sono ritenute più informative dall'algoritmo attraverso diverse inizializzazioni.
- **log_run:** viene invocato al termine di ogni esecuzione completa dell'algoritmo genetico per registrarne i risultati globali e la configurazione utilizzata. Esso riceve come input un identificativo univoco della run e un set di dati che include le prestazioni dell'individuo migliore, il tempo di calcolo totale e l'andamento di fitness e diversità lungo tutte le generazioni. Queste informazioni, insieme alle feature selezionate dal modello d'élite, vengono memorizzate nelle strutture dati del logger per consentire un'analisi statistica e comparativa della stabilità e della convergenza dell'intero esperimento.
- **summary:** Al termine dell'esecuzione dell'algoritmo, questo metodo consente di salvare i log raccolti in un formato facilmente analizzabile (nel nostro caso in .CSV).

3 Analisi dei risultati

In questa sezione analizzeremo i risultati ottenuti dalle diverse configurazioni dell'algoritmo genetico implementato, focalizzandoci sui tre scenari principali: dimensione della popolazione, operatori genetici e criteri di stop. Per ogni scenario, presenteremo i risultati in termini di convergenza, stabilità, velocità di esecuzione e robustezza della selezione. Ogni scenario è stato testato con un numero minimo di 30 run per garantire la significatività statistica dei risultati, utilizzando lo stesso seed per tutte le esecuzioni al fine di avere confronti equi. Inoltre, è stato imposto un tempo massimo di esecuzione per ogni run per evitare tempi eccessivi e garantire una gestione efficiente delle risorse computazionali. Tutti e 3 gli scenari vengono eseguiti utilizzando una funzione ausiliaria:

- **run_config:** agisce come motore esecutivo per le singole configurazioni. Essa automatizza l'esecuzione di 30 run indipendenti per ogni set di parametri, istanziando il logger e gestendo la generazione dei seed casuali per garantire la riproducibilità. Al termine delle iterazioni, la funzione aggrega i dati grezzi calcolando statistiche descrittive fondamentali, quali media e deviazione standard della fitness, del tempo di esecuzione e del numero di

feature selezionate, restituendo inoltre le curve di convergenza necessarie per l'analisi del comportamento dinamico dell'algoritmo.

3.1 Scenario Dimensione Popolazione

Lo scenario analizzato è il seguente:

- **Dimensione della popolazione:** 20, 50, 100, 200, 500
- **Crossover:** 0.8
- **Mutazione:** 0.1

L'esperimento è stato eseguito con la funzione `run_experiment_population_size` che permette di testare diverse dimensioni della popolazione, e i risultati sono stati raccolti e analizzati per valutare l'impatto della dimensione della popolazione sulla convergenza, stabilità, velocità di esecuzione e robustezza della selezione.

3.1.1 Risultati ottenuti

Di seguito i risultati ottenuti per ogni dimensione della popolazione:

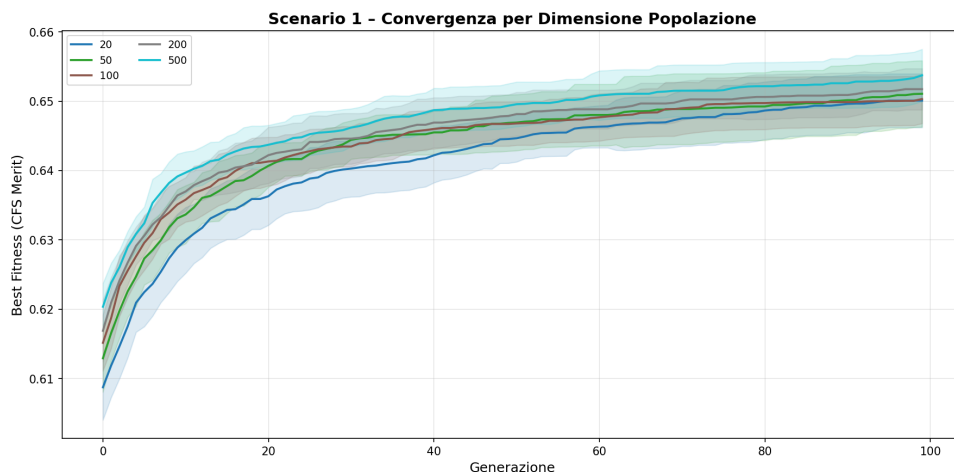


Figura 1: Convergenza per lo scenario dimensione della popolazione

Osservando le curve di convergenza, si nota come popolazioni più ampie (200 e 500 individui) presentino una fitness iniziale sensibilmente più elevata grazie alla maggiore diversità genetica presente nella prima generazione. Queste configurazioni, inoltre, mostrano una varianza ridotta (rappresentata dalle bande ombreggiate), a conferma di una ricerca più robusta e meno influenzata dalla stocasticità dell'inizializzazione. Al contrario, la popolazione da 20 individui mostra una convergenza verso valori di fitness inferiori, segno di una prematura perdita di diversità.

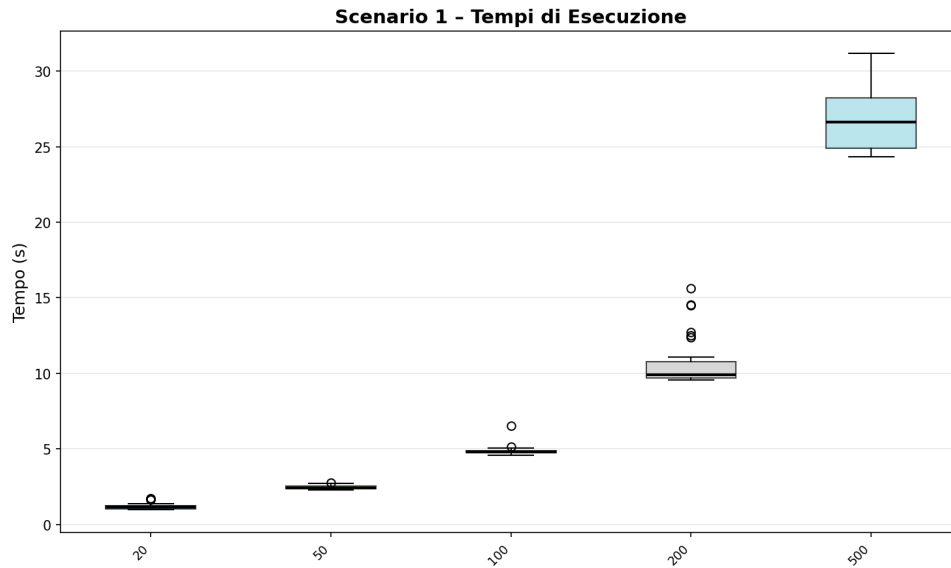


Figura 2: Tempo di esecuzione per lo scenario dimensione della popolazione

Il grafico dei tempi di esecuzione mostra una scalabilità quasi lineare del costo computazionale rispetto alla dimensione della popolazione: la run con 20 individui termina in meno di un secondo, la configurazione a 500 richiede oltre 25 secondi. Considerando che il tempo di esecuzione è un fattore critico in contesti reali, è importante bilanciare la dimensione della popolazione con le risorse computazionali disponibili. Tuttavia, i miglioramenti significativi in termini di fitness e stabilità ottenuti con popolazioni più ampie giustificano l'investimento di tempo aggiuntivo, soprattutto in scenari complessi come quello del dataset DARWIN.

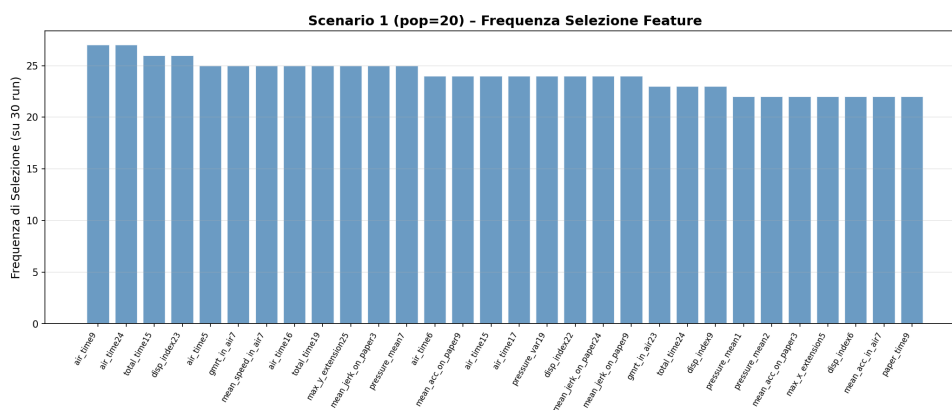


Figura 3: Frequenza di selezione delle feature per lo scenario dimensione della popolazione-20 individui

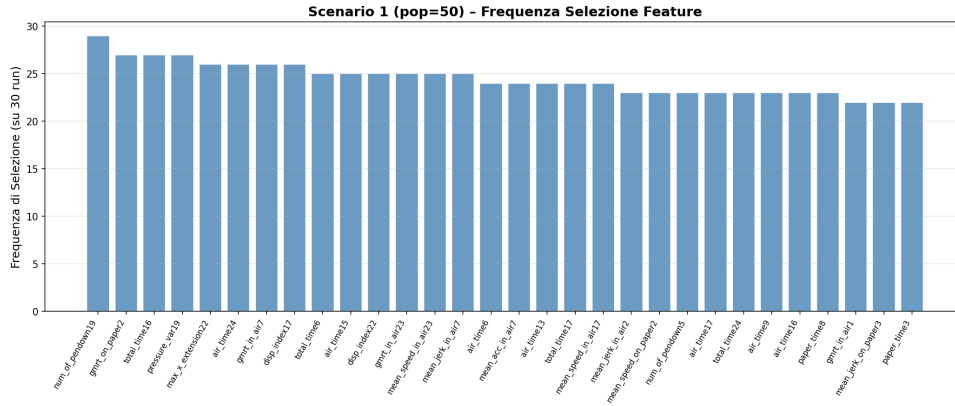


Figura 4: Frequenza di selezione delle feature per lo scenario dimensione della popolazione-50 individui

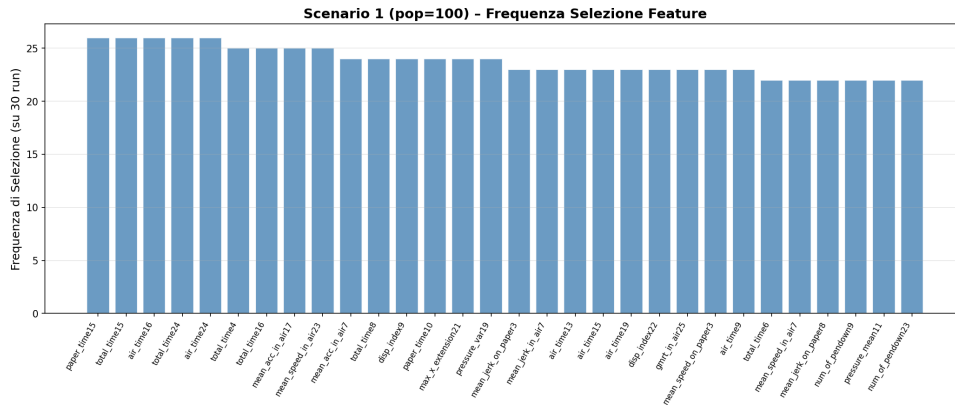


Figura 5: Frequenza di selezione delle feature per lo scenario dimensione della popolazione-100 individui

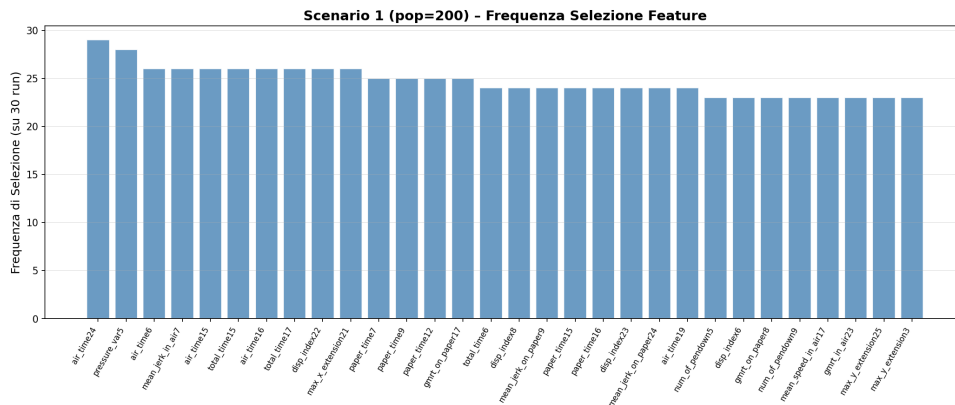


Figura 6: Frequenza di selezione delle feature per lo scenario dimensione della popolazione-200 individui

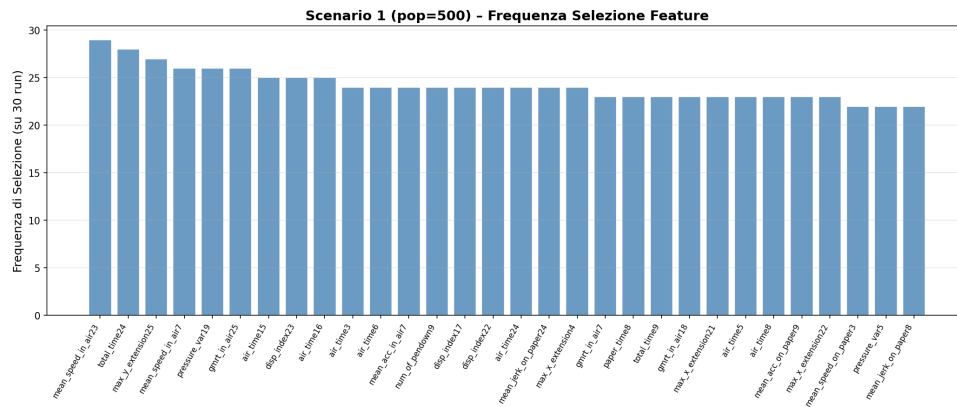


Figura 7: Frequenza di selezione delle feature per lo scenario dimensione della popolazione-500 individui

Analizzando la frequenza di selezione delle feature, si osserva che con l'aumentare della dimensione della popolazione, il GA tende a selezionare un set più diversificato di feature. Nella configurazione a 20 individui, alcune feature vengono selezionate con frequenza molto alta, indicando una possibile dipendenza da un piccolo numero di variabili. Al contrario, con 500 individui, la selezione è più distribuita tra le feature, suggerendo che il GA è in grado di esplorare meglio lo spazio delle soluzioni e identificare un set di feature più equilibrato e informativo.

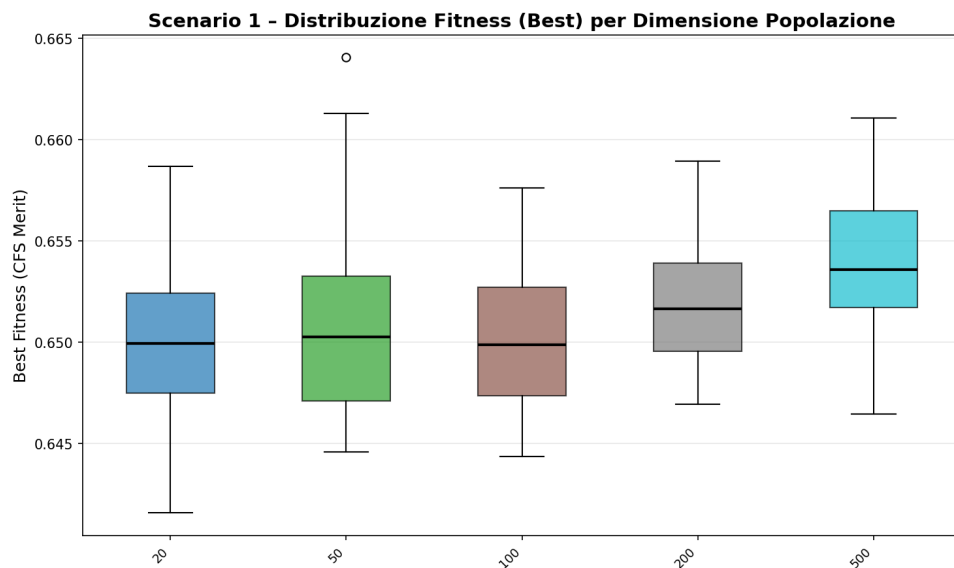


Figura 8: Fitness boxplot per lo scenario dimensione della popolazione

Analizzando la stabilità della soluzione, il boxplot della fitness finale conferma che all'aumentare della dimensione della popolazione non solo migliora il valore medio della fitness, ma si riduce drasticamente la dispersione dei risultati tra le 30 run. La configurazione a 500 individui presenta la distribuzione più compatta e i valori massimi di fitness più elevati, indicando che una popolazione numerosa è essenziale per navigare efficacemente le complesse correlazioni del dataset DARWIN senza cadere in ottimi locali.

Scenario 1 - Riepilogo:

config	best_fitness_mean	best_fitness_std	exec_time_mean	n_selected_mean
20	0.650087	0.003947	0.872637	208.066667
50	0.651073	0.004865	2.093754	207.633333
100	0.650313	0.003582	4.161101	208.033333
200	0.651737	0.003028	8.654855	208.700000
500	0.653742	0.003823	22.158297	207.866667

Figura 9: Riepilogo dei risultati per lo scenario dimensione della popolazione

3.2 Scenario operatori genetici

3.2.1 Risultati ottenuti

Di seguito i risultati ottenuti per le varie configurazioni di operatori genetici:

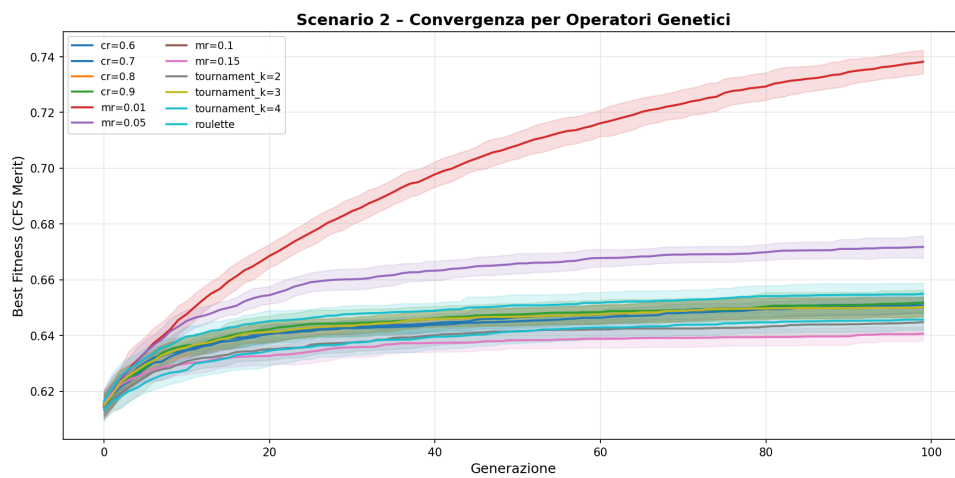


Figura 10: Convergenza per lo scenario operatori genetici

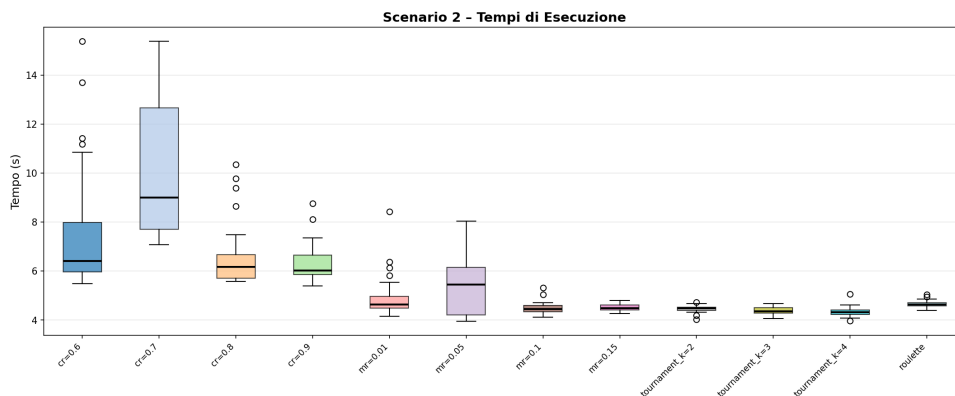


Figura 11: Tempo di esecuzione per lo scenario operatori genetici

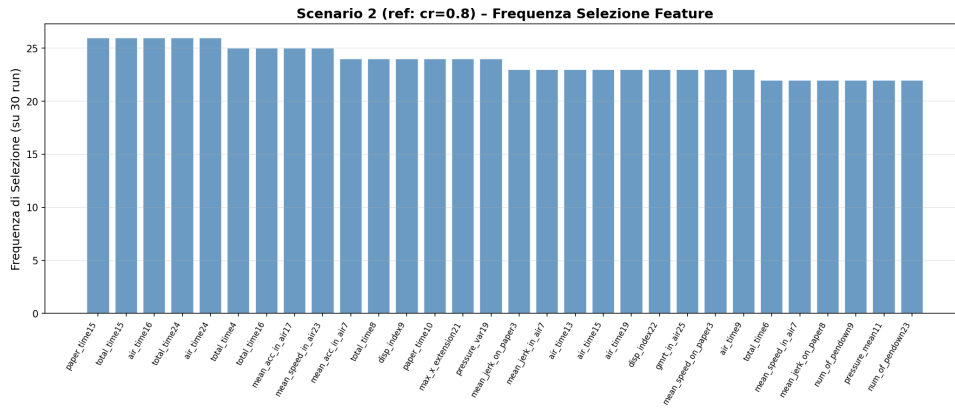


Figura 12: Frequenza di selezione delle feature per lo scenario operatori genetici

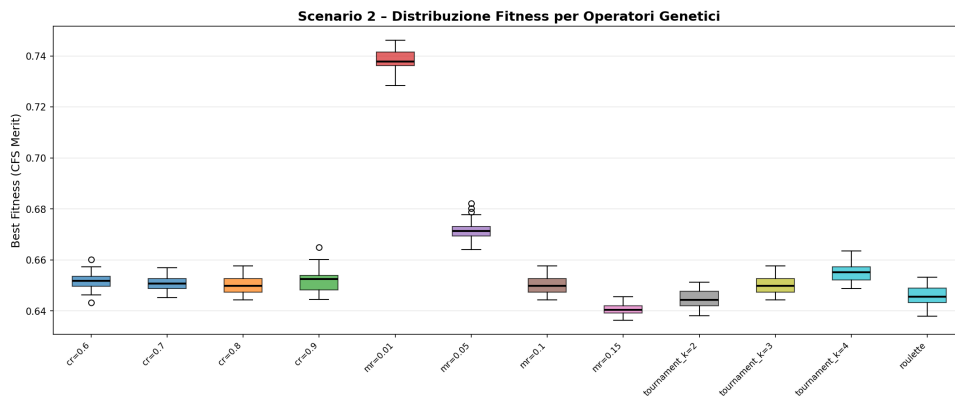


Figura 13: Fitness boxplot per lo scenario operatori genetici

Scenario 2 - Riepilogo:

config	best_fitness_mean	best_fitness_std	exec_time_mean	n_selected_mean
cr=0.6	0.651764	0.003535	4.547610	208.433333
cr=0.7	0.650840	0.002999	4.165422	210.466667
cr=0.8	0.650313	0.003582	4.346292	208.033333
cr=0.9	0.651741	0.004680	5.559775	208.100000
mr=0.01	0.738180	0.004228	3.854034	144.366667
mr=0.05	0.671795	0.004051	4.050444	195.900000
mr=0.1	0.650313	0.003582	4.272179	208.033333
mr=0.15	0.640628	0.002511	4.480010	216.700000
tournament_k=2	0.644833	0.003803	4.452146	213.300000
tournament_k=3	0.650313	0.003582	4.278998	208.033333
tournament_k=4	0.655037	0.003947	4.411805	207.433333
roulette	0.645711	0.003711	4.592597	211.533333

Figura 14: Riepilogo dei risultati per lo scenario operatori genetici

3.3 Scenario criteri di stop

3.3.1 Risultati ottenuti

Di seguito i risultati ottenuti per le varie configurazioni di criteri di stop:

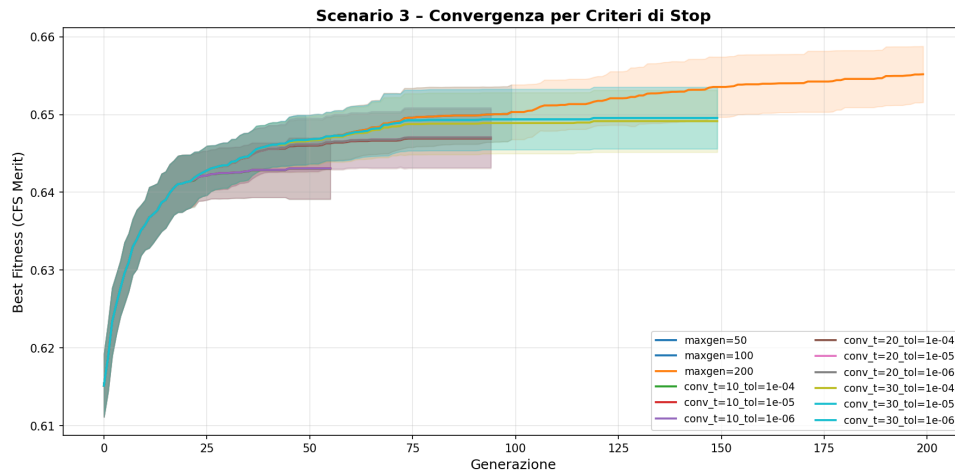


Figura 15: Convergence per lo scenario criteri di stop

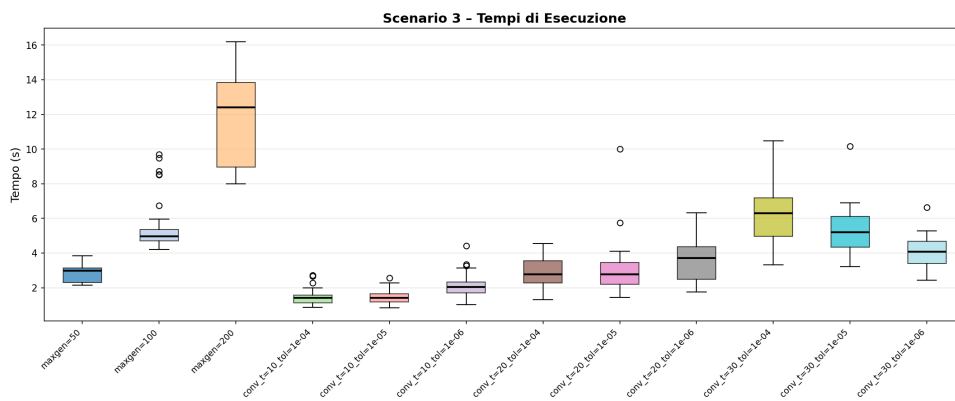


Figura 16: Tempo di esecuzione per lo scenario criteri di stop

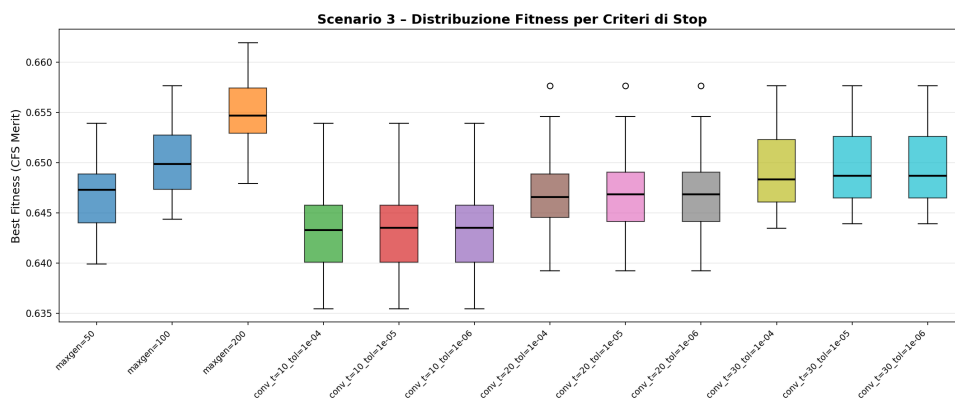


Figura 17: Fitness boxplot per lo scenario criteri di stop

Scenario 3 - Riepilogo:					
config	best_fitness_mean	best_fitness_std	exec_time_mean	gen_completed_mean	
maxgen=50	0.646718	0.003239	2.242510	50.000000	
maxgen=100	0.650313	0.003582	4.400814	100.000000	
maxgen=200	0.655173	0.003670	8.639964	200.000000	
conv_t=10_tol=1e-04	0.643042	0.004000	1.430313	30.066667	
conv_t=10_tol=1e-05	0.643070	0.004003	2.267673	30.133333	
conv_t=10_tol=1e-06	0.643070	0.004003	2.318475	30.133333	
conv_t=20_tol=1e-04	0.646901	0.003902	4.093878	57.833333	
conv_t=20_tol=1e-05	0.647054	0.003912	3.213679	58.500000	
conv_t=20_tol=1e-06	0.647054	0.003912	2.535260	58.500000	
conv_t=30_tol=1e-04	0.649147	0.004055	3.697165	86.466667	
conv_t=30_tol=1e-05	0.649558	0.004043	3.967968	87.233333	
conv_t=30_tol=1e-06	0.649558	0.004043	3.789334	87.233333	

Figura 18: Riepilogo dei risultati per lo scenario criteri di stop